• Medium • #15 • 3 Sum

▲ Problem Statement : Given an integer array "nums", find all unique triplets "[nums[i], nums[j], nums[k]]" such that :-

"nums[i] + nums[j] + nums[k] == 0"

The solution must not contain duplicate triplets.

▲ Main Problem Solution : Sort the array & use two pointers. For every element "nums[i]", find two other elements whose sum is "-nums[i]".

Use :- • left = i + 1
• right = n - 1

Since the array is sorted :- • if "sum < 0", move left.

• If "sum > 0", move right.

• If "sum == 0", store the triplet.

Skip duplicate values to avoid duplicate triplets

▲ Brute Force Method :- Use three nested loops to check every possible triplet.

Time :- O(n3)
Space :- O(1) excluding the output.

▲ Algorithm :- 1. Sort "nums"

2. Loop through each element using "i"

3. Skip "nums[i]" if it is the same as the previous element.

4. Set "left = i+1" & "right = n-1"

5. Calculate the sum of the ~~right~~ three elements.

6. If sum is "0", add the triplet & move both pointers.

7. Skip duplicate values for "left" & "right"

8. If "sum < 0", move left forward.

9. If "sum > 0", move right ~~forward~~ backward.

10] → Return all unique triplets.

▲ Important Points → • Sort the array first.

• Fix one element & use two pointers for the other two.

• Sum < 0 → move "left"

• Sum > 0 → move "right"

• Sum == 0 → store triplet

• Skip duplicates for "i", "left" & "right".

• Sorting also ensures triplets are automatically in sorted order.

▲ Complexity → Time → $O(n^2)$
Space → $O(1)$ excluding the output.